1 New Deep Reinforcement Learning

Resources

- https://github.com/alessiodm/drl-zh
- https://www.youtube.com/watch?v=2pWv7GOvuf0&list=PLqYmG7hTraZDM-OYHWgPebj2MfCFzFObQ
- https://sites.ualberta.ca/~szepesva/rlbook.html
- https://rail.eecs.berkeley.edu/deeprlcourse/resources/
- https://eecs189.org/sp26/

1. Deep Reinforcement Learning - Hands-On - M. Lapan , 2024 -- Packt, 717 pages

"Deep Reinforcement Learning Hands-On". Read up to page: 33 (xy / 717).
Links:

- PTAN: Extension to Gymnasium
- Gymnasium Documentation (link to the Gymnasium Farama web site)

1.1 Chapter 1: What is Reinforcement Learning?

The automatic learning of optimal decisions over time.

- **Supervised learning**: given pairs of input and output, automatically build a function that maps input to output
- **Unsupervised learning**: for example, clustering data
- **Reinforcement Learning** (RL):

RL: there is an agent in an environment. The agent observes the environment and it decides the actions to take. The agent receives reward from the environment. The goal of the agent is maximizing reward.
We do not want to hard-code knowledge about the environment and the actions to take, we want a method that allows out agent to learn on its own. Complications of RL:

- observations, in RL depend on the agent bahavior;
- The agent must exploit what has learned, but also exploring the environment
- Reward can be delayed after actions. Reward is a general indication of the agent performance. The agent tries to maximize accumulated reward

RL-Formalism :

- **Reward** : reward should reinforce the agent behavior in a positive or negative way. Reward is the main force that drives the agent learning process (page 11.). If training is noisy, or off course from the objective, training will go in the wrong direction.
- **Agent**
- **Environment**: everything that is outside of an agent. The agent communicates with the environment via: observations, actions and reward
- **Actions**: the things an agent can do in its environment. There are discrete and continuous actions.
  - Discrete actions: a finite set of mutually exclusive things the agent can do
  - Continuous actions: actions have one or more values attached
- **Observations**: pieces of information that the environment provides the agent to indicate what is going on around the agent

There is environment state and observations: the environment state is internal to the environment (it is not accessible to the agent), observation is the part of the environment that is accessible by the agent.

- **Policy** a setof rules that define the agent's behvior

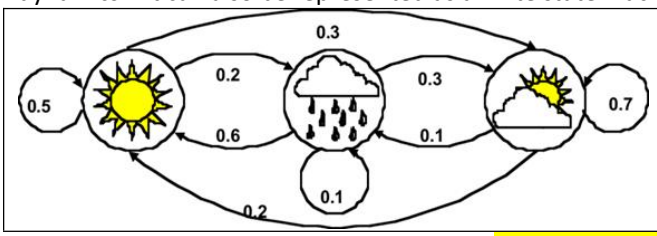
1.1.1 The Theoretical foundations of RL

1.1.1.1 The Markov Process (MP)

Markov Process = Markov Chain

You observe a system that has a set of possible states. All states form the **state space**. In MP the set of states is finite. You observe the system evolve: your observations form a sequence of states, called **Markov chain**. A sequence of observations over time is called history.

In order to call a system MP, it must satisfy the Markov property: the future system dynamics from any state depends in its state only. So the observable state is self-contained to describe the future of the system. We do not have to know the whole history to determine how the system will evolve. There is a transition matrix. If N states, the matrix is NxN. The matrix contains the probability to switch from a state into another. The matrix defines the system dynamics. It can also be represented as a finite state machine, where transitions have probabilities.



We are talking about observations, we cannot influence the weather.
In most real world situations, we only have observations, called episodes.
To get the transition matrix from the episodes, we count transitions and normalize to one.

The Markov property implies stationarity: the transition matrix values do not change over time. Non-stationarity would mean that some hidden factor influences the system dynamics (the transition matrix), but this contradicts the Markov property. The actual transitions observed in an episode are sampled from the distribution of the model.

1.1.1.2 The Markov reward process (MRP)

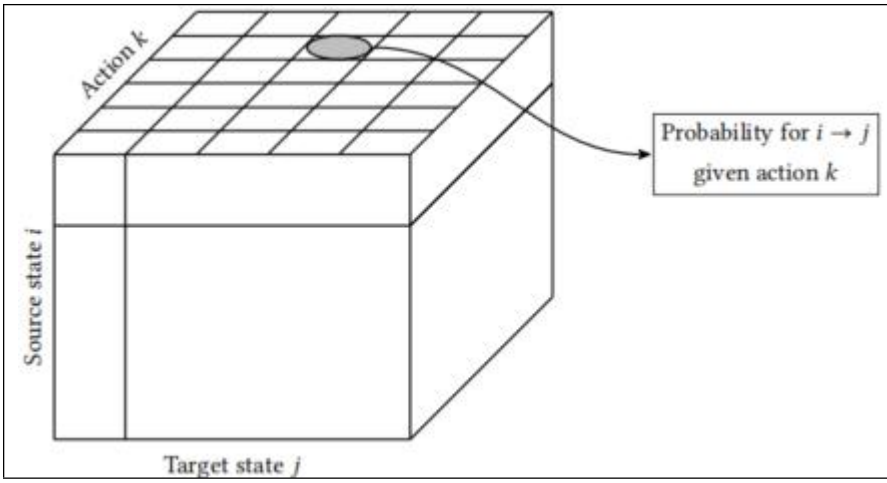
We add a value to every transition in the Transition Matrix. Reward is obtained when the state switches from i to j .
A simplified case is when reward is related to state, rather than transition: we get a more compact representation.
We add the discount factor γ "gamma", a number in the range [0, 1].
For every episode, we define **return** (gain?) at the time t as:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

In most cases, γ is 0.9 or 0.99.
The value of a state is: $V(s) = E[\, G \mid S_t = s \,]$. For every state, it is the expected return we get by following the reward process.

1.1.1.3 Adding actions to the MDP

The set of actions is finite. The Matrix becomes a cuboid of shape $|S| \times |S| \times |A|$. Previously, the transition was a matrix, each transition had a constant probability.



Now, for every action we have a different matrix, containing the transition probabilities when an action is taken. So now the cuboid has depth |A|
By choosing an action the agent can affect the probabilities of target states.

In Markov Reward Process, the reward depends on transition and action [24]
Policy: the probability of choosing a given action when the agent is in a given state.

$$\pi(a|s) = P[A_t = a \mid S_t = s]$$

[25] : *if the policy is fixed: the same actions are always chosen in the same states, then Markov Decision Process becomes Markov Reward Process, we can reduce the transition and reward matrices with a policy's probabilities and get rid of the action dimensions.*

1.2 Chapter 2: Deep Learning with PyTorch

[70] things to observe during training:

- Loss value: it has several components like base loss and regularization loss
- Result of valifdation on training and test dataset
- Statistics about gradients and weights
- Values produced by the network, depending on the problem at hand: classification / regression / etc.
- Learning rates and hyperparameters

1.3 Chapter 3: The Cross-Entropy Method

The Cross-Entropy method [87] :

- It is simple
- It has good convergence, in simple environments with frequent rewards and short episodes

Taxonomy or RL methods:

- Model free / model based: whether the method builds a model of the environment or the reward. Model free means that it connects actions to observations. Model-based methods try to predict the next observation or reward. Often making predictions to estimate more steps in the future. Based on the prediction, the method decides the best action to make. Model-based are for deterministic environments, model free for complex environments.
- Policy-based / value based. Policy-based methods directly approximate the probability distribution over the available actions. Value based methods calculate the value of every action
- On-policy: the method learns from fresh data generated from a policy that is under update. Off policy: if the model can learn from historical data

1.3.1 The Cross-Entropy method in practice

The Cross-Entropy method is [89]: model-free, policy-based and on-policy. In the Cross-Entropy Method, we replace all the agent complications with a non-linear trainable function that maps the agent input to some output. The non linear function is a Neural Network that produces the policy. The policy is $\pi(a \mid s)$: the action to take given the state. The policy is represented as a probability distribution over action. It is similar to a classification problem where the classes are the actions. The action is chosed by sampling the probability distribution.

The agent experiences episodes. An episode is a sequence of tuples, each one composed of:

- Observation
- Action
- Reward: it can be discounted or not. Let's assume $\gamma = 1$

The episode itself has a total reward. Given $\gamma = 1$ => $\sum r_i$

The cross-entropy method throws away all bad episodes and trains on the best ones.

The steps of the methods are:

- Play N episodes using the current model and environment
- Calculate the total reward for every episode and decide reward threshold (boundary). For example keep the 50th or 70th percentile-
- Throw away all episodes below the boundary
- Train on the remaining episodes using observations as input and issued actions as desired output.
- Repeat from the beginning until you are satisfied with the result

I.1. Links

- Book resources
  - gitHub repository
  - YouTube playlist
- Tools similar to Tensor Board:
  - https://mlflow.org/ : Open Source AI platform

2 REINFORCEMENT LEARNING, Z.Xiao, 2024, 576pages

Read up to page 23.
https://github.com/zhiqingxiao/rl-book : gitHub repository
https://github.com/ZhiqingXiao/rl-book/tree/master/en2024 : gitHub repository, English version of the book.

RL is a type of ML hwre the agent tries to maximize long-termrewards or minimize long-term costs. In a RL task:

- The agent observes the environment
- Executes an action
- After the action, the agent gets a reward or cost
- During the learning process, the agent learns how to maximize reward or minimize cost

In RL, the learning process is guided by the reward or cost signal.
Trying to maximize reward and minimize cost are equivalent (Michael, 1975). Reward is feedback from the environment.
Policy: of the agent makes a decision after observations.
Difference between RL and supervised/unsupervised learning. In supervised learning the correct choice is known, in RL the correct choice is unknown, the agent can only learn from reward. RL wants to maximize reward for a whole period, so it usually seeks long term superiority [3].
The agent and the environment

- On every execution step t:
  - the agent receives observation:O, executes an action A towards the environment,.
  - after the action, the environment changes its state into S_{t+1}. and sends a rewardR_{t+1} to the agent

The agent can observe both the action and the reward.
At page 6: RL systems are causal. Observations, Actions, Rewards and States are denoted by the index t

2.1.1 Taxonomy of RL (8)

- Episodic task has a start and an end, for instance the game of Go. Sequential task has no beginning and end,it processes without interruptions, forever.
- Discrete and continuous time
- Discrete action space / continuous action space. Discrete = the number of actions is countable. Discrete space does not imply finite space.
- Deterministic / stochastic environment.
- Fully observable / partially observable environment. If the environment is not observable, we cannot do anything meaninful and the problem is trivial (9)
- Multitask RL: learning for multiple tasks. If many tasks differ by the parameters of the reward function, it is called goal-conditioned RL (9).

Online RL: the agent changes during the task. Offline RL: the agent learns from recording of past interactions with the environment.
Active RL: policy optimization; passive RL: policy evaluation.
On-policy: the agent uses the latest policy and learns from that, off-policy learns from records of games with old policies.
Model-based and model-free. Model based: it uses a model of the environment. A model-based agent can also not start with a model of the environment, but learn the environment while playing. The model based agent can also generate samples from the environment model. A model free agent does not even try to model the environment.

2.1.2 Performance Metrics

“1000 papers who claim to reach State-Of-The-Art (SOTA), actually reach 1000 different SOTA(s).
The most common way to demonstrate performance is to show the learning curve (11), which is a chart.

- Long-term reward: we want large returns and episode rewards, and small standard deviation.
- The algorithm should converge to the optimal
- Sample complexity: how expansive is to receive a sample from the environment (reward and observation (?)). Better to have low sample complexity
- Regret: summation of episodes regret during the training. Episode regret is the difference between the actual episode reward and the optimal episode reward . The optimal episode reward can be defined after the traning has completed.

- Convergence speed, training time and time complexity: better if they are low.
- Occupied space and space complexity: better if they are low.

2.2 Markov Decision Process

Discrete Time MDP (DTMDP). At time t:

- The environment generates a Reward $R_t \in \mathcal{R}$ $S_t \in \mathcal{S}$

3 Mathematical Foundations of RL - Shiyu Zhao

Springer, 2025, 283 pages. Read up to page 4

4 1 - Basic Concepts

A grid world example:

- State: agent status wrt the environment.
- Actions: in the book considers that all actions are possible in every state.
- State transition occurs when the agent is in a state and selects an action: it switches into a new state.

It is possible to create a table where each row is a state and each column is an action taken in that state. In each cell is the new state . [4]. Policy [4].

4.1 Preliminaries for Probability Theory [237]

X: random variable. x:random variable value

Stochastic sequence: a sequence of random variables $\{x_i\}_{i=1}^{n}$. The random variable represents all possible values that can be sampled.

$p(X = x)$ the probability of a random variable taking a particular value.

Joint probability: $p(x, y)$ the probability that x and y take a particular value. $\sum_y p(x, y) = p(x)$

Conditional probability: $p(X = x \mid A = a)$: the probability that **X** takes **x**, given that **A** has already taken **a**.

$p(x, a) = p(x| a)\, p(a)$

Given that $p(x) = \sum_a p(x, a)$ = $\sum_a p(x|a)p(a)$: the law of total probability.

It is also considered law of total probability: $p(x|a) = \sum_y p(x, y \mid a)$

Independence between two random variables $p(x, y) = p(x)\, p(y)$

Conditional Independence: X is conditionally independent of A given B if:

$$p(X = x| A = a, B = b) = p(X = x \mid B = b)$$

Up to page 239

5 CS 285: Deep Reinforcement Learning.

Started on: 20260423.

https://rail.eecs.berkeley.edu/deeprlcourse/

https://www.youtube.com/playlist?list=PL_iWQOsE6TfXxKgI1GgyV1B_Xa0DxE5eH Playlist

http://incompleteideas.net/book/the-book-2nd.html Book

5.1 Lesson 01

The RL method produces a policy. RL is:

- a mathematical formalism for learning-based decision making.
- An approach for learning decision making and control from experience

How is RL different from other ML topics?

In ML we assume that we can learn f(x)=y. A function that, fed with x, is able to produce y. We assume that exists labelled data and the data is i.i.d independently and identically distributed. It means that an input X1 has no effect on other inputs X2, X3...., and we also assume that there is known ground truth outputs in training-

In contrast, in RL, data is not i.i.d.: previous otputs influence future inputs! Ground answer is not known , we only know if we succeeded or failed. More generally, we know the reward.

Part 2: https://www.youtube.com/watch?v=IoF7D0qec0I&list=PL_iWQOsE6TfXxKgI1GgyV1B_Xa0DxE5eH&index=2

5.1.1 Why should we care about DRL?

How do we build intelligent machines?

- Intelligent machines must be able to adapt
- DL helps handle unstructured environments. The world is unstructured and unpredictable.
- RL provides a formalism for behavior: Sequential decision making

- What is deep RL, and why should we care? In contrast to feature engineering, in DRL, end-to-end training, features engineering is done internally by the model.

Standard RL: for example the backgammon game: extract features from a backgammon game then devise a policy to select actions.
In DL, you go from the game to the action choice. End-to-end learning
What does end-to-end learning mean for sequencial decision making? End-to-end system trains at the same time the perception and the action part of the system.
Components are: actions, observations, rewards. These components are found in every RL system-
Deep models are what allow reinforcement learning algorithms to solve complex problems end to end!.
Techniques: Q-learning
Seen until lecture 1, part 03

5.2 Lecture 1, part 3:

RL maximizes rewards. Learning reward from example. We do rewards come from? Are there other forms of supervision?

- Directly copying observed behavior
- Inferring rewards from observed behavior
- Learning from observing the world: learning to predict, unsupervised learning
- Learning from other tasks: transfer learning. Meta-learning: learning to learn
- Imitation learning: the system can infer the goal, and apply the knowledge to another task

Prediction: the idea that we predict the consequences of our motor commands has emerged as an important theoretical concept in all aspects of sensormotor control
Prediction for real-world control:
Using tools with predictive models: using different tools, predicting the outcome of using the different tools.
Playing games with predictive models.

5.3 Lecture 1, part 4:

Learning as the basis of intelligence: some thinks we all do (e.g. walking). Our learning mechanism are powerful enough to do everything we associate with intelligence.
A single algorithm? An algorithm for each capability? Or a single flexible algorithm.
What must that single algorithm do? Interpret rich sensory inputs and choose complex actions. Why deep reinforement learning?
What can deep learning and RL do well now? High degree of proficency in domains governed by simple, known rules. Learn simple skill with raw sensory inputs, given enough experience. Learn from initating enough human provided expert behavior.
What has proven challenging so far? Humans can learn incredibly quickly. Deep RL methods are usually slow. While humans can reuse past experience, DRL methods have to learn from scratch. Transfer learning in RL is an open problem. It is not always clear what the reward function should be. It is not always clear what the role of prediction should be.

6 Prerequisite: Intro to Machine Learning

https://eecs189.org/sp26/
https://eecs189.org/sp26/
Book: https://issuu.com/cmb321/docs/deep_learning_ebook

6.1 Week 1, Lecture 2 : Data tools

https://www.youtube.com/watch?v=IzfaWKuxThw&list=PLuHtd0SzXhx4B8oOmp9PBEroMuV5n0MWE
Generalization vs overfitting there is always a model that behaves perfectly on training set, but it is stupid. We want the simplest model that has goog accuracy on the training data. Occam's razor: the simplest explanation that fits the data is the correct one.
Regularization: forcing the model to be simple.
ML problems: it is easy to demonstrate or evaluate the solution but difficult to directly implement it.
A human proble: the proble cannot be well specified and human judgement is required: example, determine if a message is spam. ML: learned function approximation

https://youtu.be/IzfaWKuxThw?list=PLuHtd0SzXhx4B8oOmp9PBEroMuV5n0MWE&t=1133

Supervised learning (there is X and Y)/ unsupervised learning (there is no Y)
Unsupervised: clustering, association analysis (diapers and beer), anomaly detection, generation of artificial data (create new features that look like natural ones).
Machine learning lifecycle: is a loop of [slide 69, Lecture 01]:

- Learning problem: frame a real world problem into something a machine learning model can answer
- Model design: choose the class of model, choosing the model you pick the shape of your boundary
- Optimization: fit the model
- Predict and evaluate: define a loss, evaluate the model performance on new data

https://youtu.be/IzfaWKuxThw?list=PLuHtd0SzXhx4B8oOmp9PBEroMuV5n0MWE&t=1952